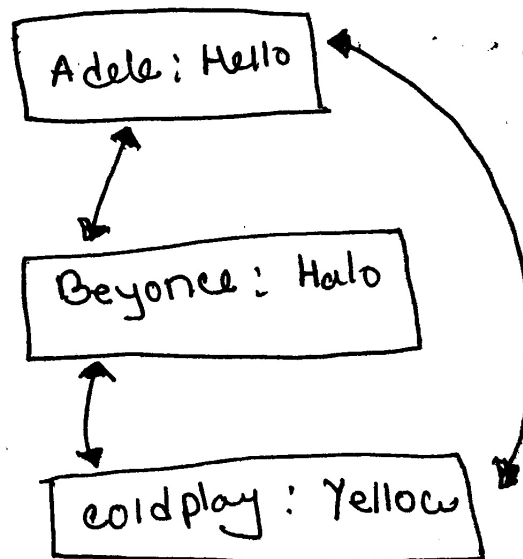


Ans to the Q. - 1

① struct Node of
char artist[50];
char song[50];
struct Node* prev;
struct Node* next;

}

②



Ans to the Q. No - 2

① void deleteNodes (struct Node** head, char* target)

struct Node* current = *head;

struct Node* temp;

while (current != NULL) {

if (strcmp (current → artist, target) == 0) {

temp = current;

current → prev → next = current → next;

current → next → prev = current → prev;

if (current == *head)

*head = current → next;

free (temp);

}

current = current → next;

⑪ void moveMiddleToFront (struct Node ** head) {

struct Node * slow = *head, * fast = *head;

if (*head = NULL || (*head) → next = NULL) return;

while (fast != NULL && fast → next != NULL) {

slow = slow → next;

fast = fast → next → next;

slow → prev → next = slow → next;

if (slow → next) slow → next → prev = slow → prev;

slow → next = *head;

slow → prev = NULL;

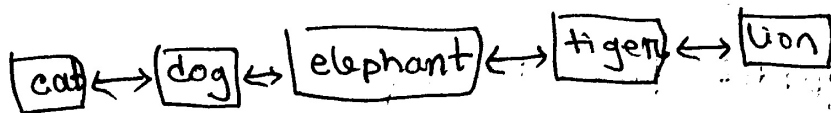
(*head) → prev = slow;

*head = slow;

}

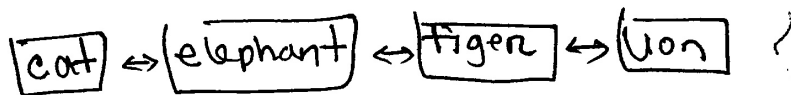
①

initial doubly list



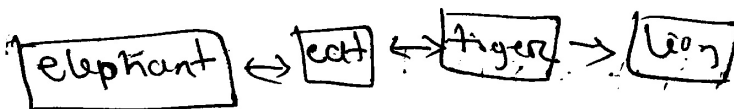
②

After deleting node with value "dog"



③

After moving middle ele of the front



Ans to the Q.No - 3:

```
typedef struct Node {
```

```
    int value;
```

```
    struct Node* next;
```

```
};
```

```
void swapFirstLast (Node** head) {
```

```
    if (*head == NULL || (*head) -> next == NULL)
```

```
        return;
```

```
    Node *first = *head, *last = *head, *prevLast  
        = NULL;
```

```
    while (last -> next) {
```

```
        prevLast = last;
```

```
        last = last -> next;
```

```
    }
```

```
    if (first == last) return;
```

int temp = first → value;

first → value = last → value;

last → value = temp;

return;

}

void reverseStack (int* stack, int* top)

{

Ans to the Q NO-4

void emptyStack (int* stack, int* top) {

while (*top > 0) {

printf ("%d\n", pop (stack, top));

}

}

Ans to the Q. no - 5

Queue size = 7

① Queue size = 7

Front = 3; rear = 5

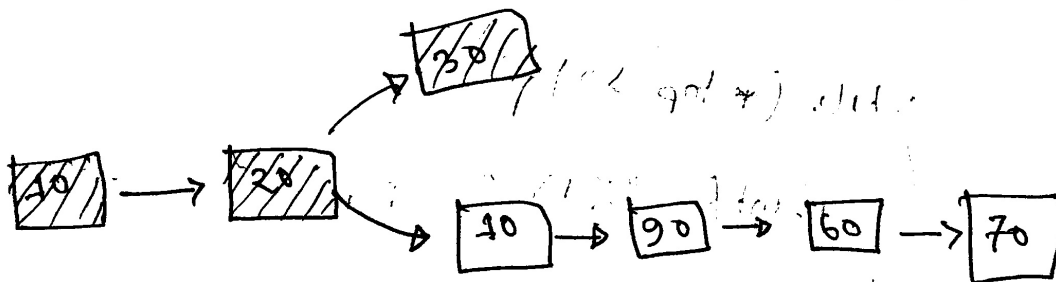
element start from index 3 to rear index 5

9, 10, 2 so elements total : 3

1 - element will not delete

②

queue size = 6, front = 20, rear = 70



(iii) two pointers, would be required; one for the front and one for the rear of the queue. These allow efficient insertion and deletion operations.

(iv) I agree, A linked list based queue can dynamically allocate memory allowing it to grow as needed (within the system memory) and avoiding overflow issues typical in fixed size array implementations.

Ans to the Q.NO - 61

```
void deletemin (int s1[]) {
```

```
    int min = s1[top], min_pos = top;
```

```
    int s2[5], top2 = -1;
```

```
    while (top > -1) {
```

```
        if (s1[top] < min) {
```

```
            min = s1[top];
```

```
            min_pos = top;
```

```
        }
        s2[++top2] = s1[top--];
```

```
    for (int i = 0; i <= top2; i++) {
```

```
        if (i != min_pos) s1[++top] = s2[i];
```